Karpathy-LLM-Wiki-System-Briefing

Karpathy LLM Wiki System — Complete Briefing

What We Built

Two AI-powered knowledge bases that automatically turn everything you read, save, and discuss into a searchable, interlinked wiki — like having a research librarian who works 24/7 organizing your brain.

Research Wiki — Everything you learn from the outside world. Articles you clip, YouTube videos you watch, Substack newsletters, Recall saves. Your external knowledge organized by concept, not by date.

Personal Wiki — Everything that happens inside your business. Meeting transcripts, AI conversations where you made decisions, journal entries, Slack threads. Your institutional memory.

Why It Matters

The Problem It Solves

You consume massive amounts of information. Articles, videos, meetings, conversations — probably 50-100 pieces of content per week. Right now that content goes into folders where it sits. You might remember "I read something about X" but you can't find it, can't cross-reference it, and certainly can't ask questions across all of it.

What Changes

Instead of content sitting in folders, Gemma 4 (running locally on your machine, no API cost) reads every piece of content and creates wiki pages for every person, concept, decision, project, and tool mentioned. It links them together with backlinks. So when you open the vault and ask Claude Code

"what do my sources say about AI agents for business?" — it reads the index, navigates to the relevant concept pages, follows the backlinks to specific sources, and gives you a synthesized answer across everything you've ever saved on that topic.

How You Actually Use It

Day-to-day, you don't do anything different. You keep clipping articles, saving to Recall, having meetings. The sync daemons pull new content in automatically. Gemma 4 processes it in the background.

When You Need It

1. Open the vault in Obsidian (you can browse visually — graph view shows all connections)
2. Open a terminal and run `claude` from that vault directory
3. Ask questions:
   - "What have I read about [topic]?" (Research Wiki)
   - "What decisions have I made about [project] in the last 3 months?" (Personal Wiki)
   - "Who has been involved in [initiative] and what were their positions?" (Personal Wiki)
   - "What frameworks have my sources recommended for [problem]?" (Research Wiki)
   - "What did we discuss in meetings about [topic] and what action items came out of it?" (Personal Wiki)

Why This Is Different From What You Had Before

Before: 23,000+ files across multiple vaults. To find something, Claude Code had to search, grep, read files one by one. Expensive in tokens, slow, often missed connections.

Now: A master index tells Claude Code exactly where to look. Wiki pages pre-summarize content and link related topics. Claude Code reads the index (one file), navigates to 3-5 relevant pages, and answers. Karpathy's claim of 95% token reduction is real — instead of reading 50 raw files, it reads 5 wiki pages that already synthesize those 50 files.

How To Talk About It

The One-Liner

"I have a local AI model that automatically turns everything I read and every meeting I have into an interlinked knowledge base I can query in natural language."

The Elevator Pitch

"Andrej Karpathy — founding member of OpenAI, former head of AI at Tesla — published a pattern for using Obsidian as a lightweight alternative to vector database RAG systems. I implemented it with two wikis: one for everything I learn externally, one for my business history. A local model runs 24/7 processing content into interlinked wiki pages. When I need to recall something, I ask Claude Code and it navigates the wiki in seconds instead of searching through thousands of raw files. Zero API cost for the processing."

The Strategic Framing

"Most people's knowledge disappears after they consume it. They read an article, have a meeting, make a decision — and six months later they can't find any of it. This system makes knowledge compound. Every article I save makes every future article more connected. Every meeting transcript adds context to every past decision. The more I use it, the smarter it gets."

Why Karpathy Specifically Matters

"This isn't some random hack. Karpathy is one of the most respected AI researchers alive. When he says you don't need expensive vector databases and complex RAG infrastructure — that a simple folder of markdown files with a good index beats enterprise search for 90% of use cases — people listen. And he's right. The LLM itself is the retrieval engine. You just need to give it a map."

The Technical Architecture (Simple Version)

Two Folders and an Index

```
wiki-vault/
├── raw/        ← Content dumps here automatically
├── wiki/       ← AI creates interlinked pages here
│   ├── index.md ← The "map" — Claude reads this first
│   ├── people/  ← One page per person
│   ├── concepts/← One page per idea/framework
│   ├── sources/ ← One page per article/transcript
│   └── ...
└── CLAUDE.md    ← Navigation rules for AI
```

The Flow

1. You clip an article / have a meeting / save something to Recall
2. Sync daemon copies it to `raw/` within 5 minutes
3. Gemma 4 (local model, zero cost) reads it and creates wiki pages
4. Wiki pages link to each other with `[[backlinks]]`
5. When you query, Claude Code reads the index first, navigates to relevant pages, synthesizes an answer

What's Running

- **2 sync daemons** — pull new content every 5 minutes from all your Obsidian vaults, Recall exports, YouTube transcripts, Substack articles
- **2 ingest daemons** — Gemma 4 E4B processes raw files into wiki pages 24/7
- **Zero API cost** — Gemma 4 runs locally on the M3 Ultra

Where It Replaces Traditional RAG

	Traditional RAG	Karpathy Wiki Pattern
Infrastructure	Vector database + embedding model + chunking pipeline	Markdown files in a folder
Cost	Ongoing compute + storage	Free (local model)
Setup	Complex, technical	5 minutes
How it finds info	Similarity search (guesses)	Index + backlinks (navigates)
Relationships	Implicit (chunks that "seem similar")	Explicit (linked wiki pages)
Maintenance	Re-embed when content changes	Run a lint, add more content
Scales to	Millions of documents	Hundreds to low thousands
Best for	Enterprise, massive datasets	Solo operators, small teams

---

Use Cases Where This Pattern Is Optimal

Research and Analysis

- **Research synthesis** — 50 papers on a topic become interlinked concept pages you can query across

- **Competitive intelligence** — Competitor content, pricing, press releases mapped into a queryable landscape
- **Market landscape mapping** — Ingest everything about an emerging space, the wiki surfaces connections

Content and Knowledge Management

- **Content library indexing** — Every video, podcast, article indexed by concept with cross-references
- **Expert knowledge capture** — Interview transcripts turned into queryable frameworks and insights
- **Book/course writing research** — All source materials organized with backlinks and citations

Business Operations

- **Institutional memory** — Every meeting, decision, and conversation becomes searchable forever
- **Client intelligence** — All communications organized by person, concern, and timeline
- **Team onboarding** — New person asks "how do we handle X?" and gets an answer from the wiki

Personal and Strategic

- **Personal second brain** — Life context organized by concept, not date
- **Strategic planning** — Industry reports + internal metrics + customer research in one queryable system
- **Investment research** — Each asset's research interlinked, macro themes surfaced across holdings

---

What It Will Look Like In 30 Days

Once both daemons have processed a few thousand files:

- The Obsidian graph view will show clusters of interconnected topics
- People pages will have references across dozens of meetings and conversations
- Project pages will have full timelines of decisions and discussions
- You'll be able to ask questions that span years of content and get sourced answers
- New content you save will automatically connect to everything that already exists

The value compounds over time. Day 1 it's a system. Day 30 it's a knowledge advantage no one else in your space has.

Our Implementation Stats

	Research Wiki	Personal Wiki
Raw files	13,137	10,079
Sources	Clippings, Recall, YouTube, Substacks	Conversations, Limitless, Journals, Slack
Processing model	Gemma 4 E4B (local)	Gemma 4 E4B (local)
Processing speed	~4 min/file	~4 min/file
Full processing time	~40 days (background)	~30 days (background)
API cost	$0	$0

Key Insight

The internet went viral over this because Andrej Karpathy — arguably the most credible voice in AI — said the quiet part out loud: **you don't need the complex infrastructure everyone's been selling you.** Folders, markdown, and a smart index beat vector databases for 90% of real-world use cases. The LLM itself is the retrieval engine. You just need to give it a map.